



Documentación del Proyecto

AI-CodeAuditor

Elaborado por: Roberto Alejandro Chagra Martínez

Guía Paso a Paso del Desarrollo Técnico

1. Diseño de Arquitectura y Configuración del Entorno Inicial

- **Visión del Sistema:** Diseñamos una arquitectura híbrida que inicialmente funcionara "On-Premise" para garantizar la privacidad del código corporativo, pero estructurada en microservicios para escalar fácilmente a SaaS en la nube.
- **Selección de Stack Tecnológico:**
 - **Frontend:** Next.js (React) con TypeScript y TailwindCSS para interfaces dinámicas (SSR/CSR).
 - **Backend:** FastAPI (Python 3.11) por su velocidad y manejo nativo de asincronía.
 - **Base de Datos y Caché:** PostgreSQL 15 gestionado mediante SQLAlchemy (ORM basado en Pydantic y SQLAlchemy) y Redis 7 como Message Broker.
 - **Orquestación:** Celery para encolar trabajos en segundo plano y Docker Compose para levantar el stack completo.

2. Desarrollo del Backend API (FastAPI) y Persistencia

- **Gestión de Base de Datos (core/database.py):** Configuramos una conexión transaccional robusta hacia PostgreSQL usando create_engine de SQLAlchemy.
- **Modelado de Datos (models/audit.py):** Definimos el modelo AuditJob usando SQLAlchemy que hereda propiedades persistentes en la BD. Creamos campos críticos: id, repository_name, pr_number, status (PENDING, IN_PROGRESS, COMPLETED, FAILED), findings (tipo JSON) y un created_at automático.
- **Enrutamiento y Endpoints (api/audits.py y api/webhooks.py):**
 - **GET /api/audits/:** Diseñado para que el Dashboard lea todas las auditorías previas de la tabla AuditJob.
 - **POST /api/webhooks/github:** Endpoint síncrono encargado de interceptar cargas útiles de repositorios (Payloads de PRs). En

lugar de auditar el código al instante (lo cual causaría un Timeout en GitHub), inserta la orden en una tabla y dispara a Celery asincrónicamente mediante `dummy_audit_task.delay()`.

- **POST /api/audits/manual:** Endpoint para auditorías directas vía texto plano en la aplicación web para un feedback en vivo, ejecutando la inferencia IA síncronamente y no almacenando en BD.

3. Integración Avanzada de la Inteligencia Artificial (Llama 3 Local)

- **Motor On-Premise (services/llm.py):**
 - Implementamos LangChain con la librería nativa langchain_ollama configurando el

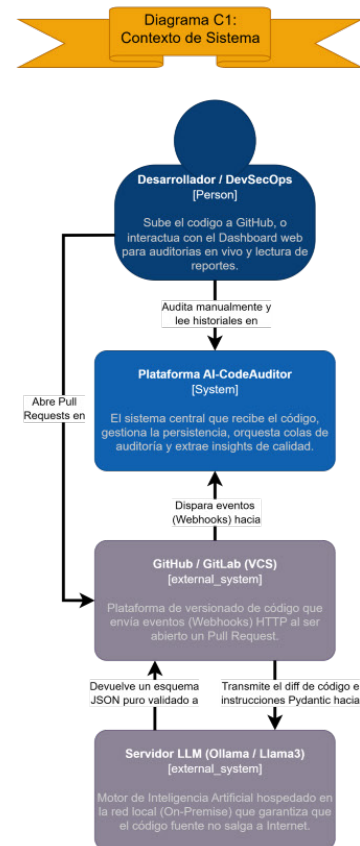


Imagen 1.
Diagrama C1: Contexto de Sistema

modelo en temperature=0.1 para asegurar respuestas deterministas (poca alucinación) óptimas para análisis de código.

- Para evitar problemas de redes aisladas en Docker, enrutamos el LLM a <http://host.docker.internal:11434> comunicando al contenedor con el servidor local de Windows.

Validación Estricta con Pydantic y JSON Output Parsers:

- Aprovechando `langchain_core.output_parsers.JsonOutputParser` y una clase base Pydantic (`AuditResult`), obligamos a la IA a seguir una plantilla estricta de estructura.
- Refinamos el System Prompt explícitamente solicitando la prohibición de texto introductorio conversacional (el típico "Aquí tienes la respuesta...") usando el parámetro nativo `format="json"` en el motor.
- Se forzó el idioma español en las directrices maestras del System Prompt.

Manejo de Errores (Fallback Mock):

- Añadimos un bloque `try-except` que, si detecta fallos en la conexión o bloqueos de parseo, retorna un payload "simulado" con

Diagrama C2: Contenedores del Sistema

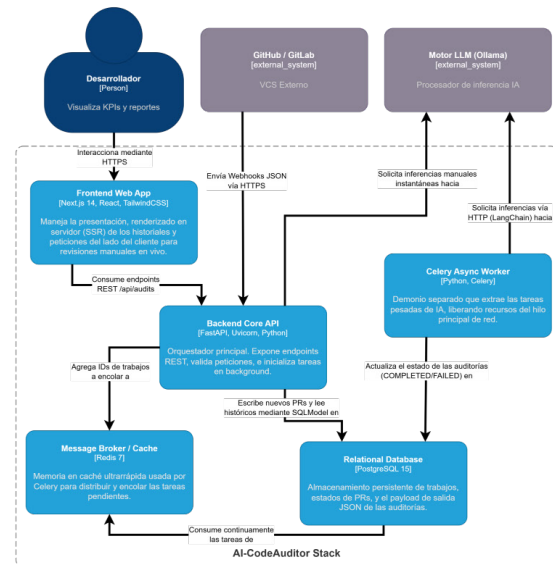


Imagen 2.
Diagrama C2: Contenedores del Sistema

avisos sobre la desconexión del LLM, protegiendo al Frontend de excepciones críticas (HTTP 500).

4. Desarrollo del Frontend (Next.js 14+)

UX/UI "DevOps Premium":

- Implementamos un diseño oscuro moderno (Dark Theme) apoyado en un Glassmorphism de alto contraste y tarjetas minimalistas.

Componentes de la Página Principal (`src/app/page.tsx`):

- Construimos un Dashboard renderizado en servidor (Server Components) que hace un "fetch" asíncrono a nuestro servicio backend `http://localhost:8000/api/audits`.
- El dashboard interpreta las cargas JSON crudas de la BD e imprime el estatus de forma colorizada y dinámica (por ejemplo: mostrando insignias de peligro si las vulnerabilidades de seguridad son mayores a cero).

Visor de Inferencia Manual (`src/app/audits/page.tsx`):

- Creamos un formulario interactivo en el cliente ("use client") donde se puede pegar

Diagrama C3: Componentes Internos del Backend

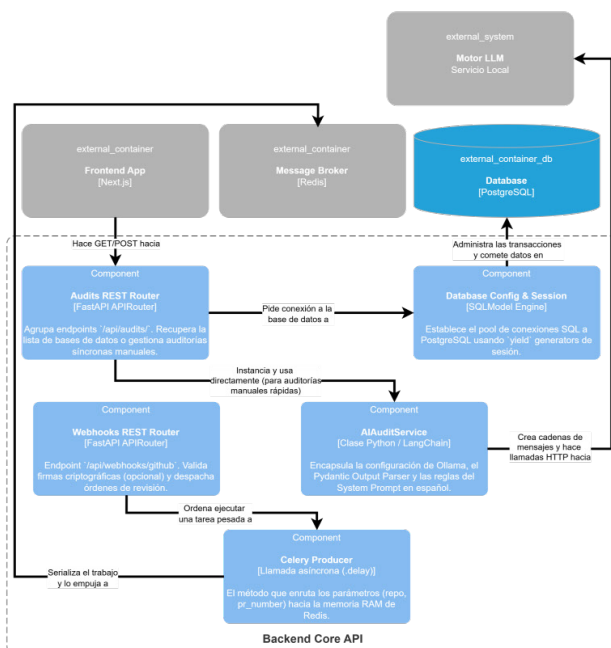


Imagen 3.
Diagrama C3: Componentes Internos del Backend

Diagrama C4:
Código / Modelo de Objetos

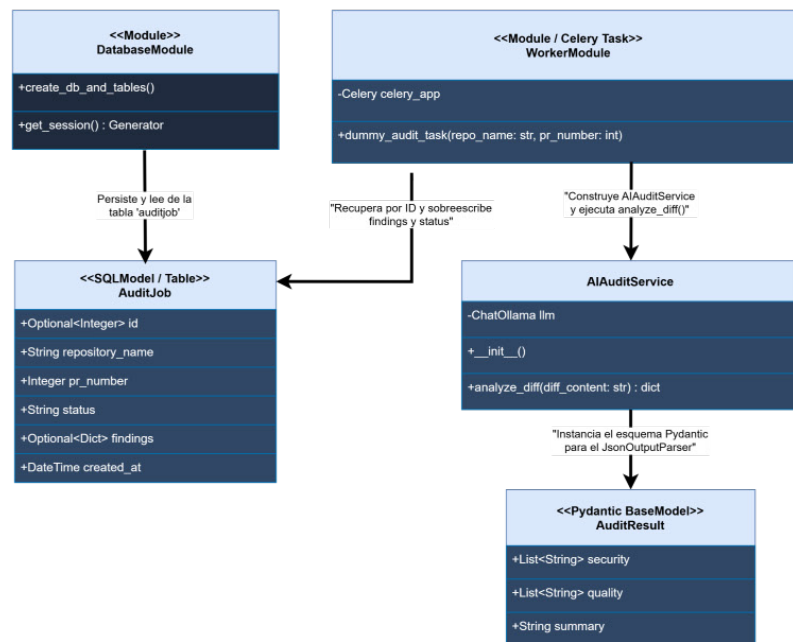


Imagen 4.
Diagrama C4: Código / Modelo de Objetos

- Se conectó mediante fetch directamente hacia `/api/audits/manual`, reflejando las respuestas "Quality" y "Security" de LangChain de inmediato a través del uso de Estados React (`useState`).

5. Orquestación y Empaquetado Final (Docker & Celery)

• Docker Compose (`docker-compose.yml`):

- Empaquetamos la aplicación en múltiples contenedores aislados: db (Postgres), redis (Broker de mensajería), backend (FastAPI) y `celery_worker`.
- Se utilizó `uvicorn --reload` en conjunto con mapeo de volúmenes en Docker, lo que posibilitó el desarrollo en vivo (Hot-Reload) sin tener que compilar contenedores en cada cambio de archivo Python.

• Worker Asíncrono (`worker.py`):

- Definimos un entorno de Celery ligado al Redis contenedorizado.
- Añadimos la tarea base `dummy_audit_task` que (en el entorno completo) extrae el diff de GitHub utilizando el PR y ejecuta el motor LLM independientemente del servidor de red principal.